

Python 专项提升学习计划

6 周可执行路线 | 面向 Python 初级到中级面试

你的学习定位

根据本轮模拟面试：你已有字典统计、enumerate、排序键和基础引用概念的能力；当前要优先补强代码执行推演、概念表达的准确性、函数参数机制与复杂度意识。

周期	投入建议	输出成果
6 周	每天 60 至 90 分钟；每周 1 天复盘	一套错题本、45 道题的答案、3 次阶段测验、1 次最终模拟面试
训练比例	40% 执行流程；30% 核心概念；30% 编程题与算法	能写、能讲、能用边界条件验证

使用方式：每天先完成“学习任务”，再独立完成代码练习；不要先看答案。答错的题写入第 6 周提供的错题模板。每周末完成自测并用评分标准复盘。

目录

本计划可从头执行，也可以先阅读“本轮面试复盘”，再按周推进。页码以 PDF 页脚为准。

章节	内容
A	学习地图与本轮 5 题面试复盘
第 1 周	代码执行流程分析：循环、分支、返回与边界
第 2 周	变量、对象、引用、可变性与复制
第 3 周	函数、参数、作用域与默认参数陷阱
第 4 周	面向对象：self/cls、方法、属性、继承
第 5 周	复杂度、哈希、双指针、滑动窗口、栈队列
第 6 周	二分、递归、DFS/BFS、排序、贪心、DP 入门与冲刺
题库	20 道选择题、15 道简答题、15 道编程题：均附答案与解析
工具	阶段测验、评分标准、错题记录、打卡表、最终模拟面试清单

6 周总览

周次	主线	达到的面试能力
1	读代码与控制流	能逐行推演，解释 return 放置位置
2	对象模型与复制	能精确说明引用、浅拷贝和深拷贝
3	函数与作用域	能解释默认参数、*args/**kwargs 与 LEGB
4	面向对象	能比较实例/类/静态方法并写出继承代码
5	复杂度与基础结构	能估算 O()，选择哈希、双指针或滑窗
6	常见算法与整合面试	能完成基础算法题并表达取舍

A. 本轮模拟面试复盘

目标不是记住标准答案，而是把“为什么”说清楚。以下五题对应本计划的训练主线。

第 1 题：引用与浅拷贝

- 本次表现：表现：正确。a 与 b 指向同一列表；c 是新的外层列表。
- 下一步：升级点：浅拷贝只复制外层。若元素包含嵌套列表，修改嵌套对象仍会互相影响。

第 2 题：可变默认参数

- 本次表现：表现：发现了复用隐患，但没有推演实际输出。
- 下一步：关键句：默认参数在函数定义时创建一次，不是每次调用创建。可变默认值使用 None 哨兵。

第 3 题：三种方法

- 本次表现：表现：概念表达不准确。
- 下一步：答题框架：实例方法接收 self，处理对象状态；类方法接收 cls，处理类状态/替代构造；静态方法不自动接收二者，是归属于类的工具函数。

第 4 题：质数函数

- 本次表现：表现：发现了 `number < 2`，但遗漏循环中的提前 return。
- 下一步：训练动作：用 9 手动执行。i=2 时不能整除，若 else 内 return True，程序会提前结束，错判 9。

第 5 题：出现最多元素

- 本次表现：表现：正确且使用了合理的排序键。
- 下一步：升级点：先确认并列规则；空列表的行为；分析排序带来的 $O(k \log k)$ ，并能给出 $O(n)$ 遍历改进。

固定答题顺序

读题先确认输入、输出、并列规则与异常约定；读代码先列变量状态，再逐轮写出循环值；最后用边界案例验证。回答时先给结论，再给机制，最后举一个最小例子。

第 1 周：代码执行流程分析

把“看起来对”变成“能证明对”。训练逐行推演、循环边界与返回位置。

本周知识点

- for/while 的初始化、条件、更新与终止
- if/elif/else 缩进及分支覆盖
- return、break、continue 的不同控制范围
- for/while 的 else：仅在未被 break 打断时执行
- 手动推演表与边界测试：空、单元素、临界值、异常值

每日执行任务

日期	主题	完成动作
Day 1	for 与 range	手写 range(2, 6) 的 i 序列；阅读 5 段累加代码，逐步记录变量。
Day 2	while 与终止	为 3 个 while 写出循环不变式和终止条件；修复 2 个无限循环。
Day 3	缩进与分支	比较 return 在 if 内、循环内、循环外的差异；用 9 测试质数函数。
Day 4	break/continue	分别写跳过偶数、找到即停止、处理完再返回三个小程序。
Day 5	循环 else	写搜索目标的 for...else；解释什么时候 else 不执行。
Day 6	边界测试	为求最大值、求质数、去重写测试用例：空、1 个、重复、负数。
Day 7	周复盘	完成本周自测；把所有错题按“缩进/边界/提前返回”分类。

代码练习

- 修复 is_prime：先处理 number < 2；循环只要发现因子就 False；循环结束才 True。
- 实现 find_first(nums, target)：找到返回下标；找不到返回 -1；用 for...else。
- 实现 sum_until_negative(nums)：遇到负数停止，返回此前元素之和。

面试问答卡

问题	最佳回答要点
为什么循环里的 else 常被误用？	它不等价于 if 的 else；只有循环自然结束、没有执行 break 才运行。
return 和 break 有何区别？	return 立即结束整个函数并可带回结果；break 只结束当前一层循环，函数仍继续执行。

易错点

- 在第一次不满足整除时就 return True。
- while 未更新计数器导致死循环。
- 忘记空列表、0、1、负数等边界输入。

本周自测与参考答案

- 题目：判断 9 是否为质数时，为什么不能在 i=2 不能整除后直接 return True？
参考答案：9 还可能被 3 整除；必须检查完所有候选因子。
- 题目：for 循环的 else 在什么情况下不执行？

参考答案：循环内部执行了 `break` 时不执行。

3. 题目：对 `nums=[]` 求最大值，函数应先考虑什么？

参考答案：约定空输入行为，如返回 `None` 或抛出 `ValueError`，并在访问前判断。

周末交付物

提交 1 页错题复盘：错误现象、最小反例、根因、正确规则、重新写出的代码或口述答案。每周至少完成 3 道代码题并写出时间/空间复杂度。

第 2 周：变量、对象、引用与复制

准确说清“变量保存的是对象引用”，并能用最小示例验证可变性与复制层级。

本周知识点

- 变量名、对象、id 与引用关系
- 可变类型：list/dict/set；不可变类型：int/str/tuple
- 赋值、浅拷贝 copy()/[:]/、深拷贝 copy.deepcopy
- 嵌套可变对象的共享风险
- == 比较值；is 比较对象身份

每日执行任务

日期	主题	完成动作
Day 1	对象与引用	运行 a=b、修改 b 的例子；用 id() 观察对象身份。
Day 2	可变/不可变	分别修改列表、字符串、元组；解释为何 str 的“修改”创建了新对象。
Day 3	浅拷贝	比较 a.copy()、a[:] 和 list(a)，对一维与二维列表分别测试。
Day 4	深拷贝	用 copy.deepcopy 处理嵌套列表；画出外层与内层引用。
Day 5	is 与 ==	写 8 个判断预测结果；区分内容相同与同一对象。
Day 6	真实场景	写函数时避免原地修改调用者数据；必要时复制并在注释中声明。
Day 7	周复盘	完成阶段测验 1 的对象模型部分，口述 3 分钟解释浅/深拷贝。

代码练习

- 1 构造二维列表 matrix，分别演示浅拷贝和深拷贝后修改 matrix[0][0] 的结果。
- 2 实现 clone_config(config, deep=False)，用参数明确复制策略。
- 3 修复 rows = [[0]*3]*3 导致的行共享问题。

面试问答卡

问题	最佳回答要点
为什么 b=a 后 append(b) 会影响 a？	赋值没有复制列表；两个变量绑定同一可变对象。
浅拷贝什么时候足够？	容器元素都是不可变对象，或不需要独立修改嵌套对象时。

易错点

- 把“浅拷贝”说成“完全不共享”。
- 用 is 比较字符串或整数的值。
- 用 [[0]*m]*n 创建二维数组，误共享同一行。

本周自测与参考答案

1. 题目：a=[[1],[2]]；b=a.copy()；b[0].append(9)，a 是什么？
参考答案：[[1, 9],[2]]，因为内层列表仍共享。
2. 题目：== 与 is 分别比较什么？
参考答案：== 比较值是否相等；is 比较是否同一对象。

3. 题目：如何避免二维列表行共享？

参考答案：用 `[[0]*m for _ in range(n)]`，每次创建新行。

周末交付物

提交 1 页错题复盘：错误现象、最小反例、根因、正确规则、重新写出的代码或口述答案。每周至少完成 3 道代码题并写出时间/空间复杂度。

第 3 周：函数、参数与作用域

能推演函数调用并解释参数绑定、None 哨兵、*args/**kwargs 和 LEGB。

本周知识点

- 位置参数、关键字参数、默认参数
- 可变默认参数陷阱与 None 哨兵
- *args、**kwargs 的收集与解包
- 局部变量、global、nonlocal 与 LEGB
- 函数返回值、None 与副作用

每日执行任务

日期	主题	完成动作
Day 1	参数绑定	用同一函数分别采用位置与关键字调用；识别重复赋值引发的 TypeError。
Day 2	默认参数	推演 add_item 连续调用的四行输出；改写为 None 哨兵。
Day 3	*args/**kwargs	写 total(*nums) 和 format_user(**info)；练习列表、字典解包。
Day 4	LEGB	画出 Local/Enclosing/Global/Builtins 查找顺序；预测闭包变量输出。
Day 5	返回与副作用	比较 print、return 与原地修改；为函数写明确的返回约定。
Day 6	函数设计	把长函数拆为校验、计算、格式化三个纯函数。
Day 7	周复盘	完成自测，口述“为什么默认列表只创建一次”。

代码练习

- 1 实现 add_item(item, container=None)，保证每次未传 container 时返回独立列表。
- 2 实现 merge_scores(*score_lists)，返回所有数字的总和与平均值；空输入返回 (0, None)。
- 3 实现 apply_discount(price, *, rate=0.9)：强制 rate 用关键字传入并校验范围。

面试问答卡

问题	最佳回答要点
为什么用 if container is None 而非 if not container？	空列表是合法传入值；if not container 会误把它当作未传入。
LEGB 的顺序是什么？	Local、Enclosing、Global、Builtins。名称从内到外查找。

易错点

- 写 def f(x=[]) 并认为每次调用都会创建列表。
- 用 mutable 默认值缓存却没有明确说明意图。
- 忘记函数没有 return 时默认返回 None。

本周自测与参考答案

1. 题目：add_item(1)、add_item(2) 的共享默认列表为什么累积？
参考答案：默认列表在 def 执行时创建一次，两个调用绑定到同一对象。
2. 题目：何时使用 *args？

参考答案：调用者传入的位置参数数量不固定时收集为元组。

3. 题目：nonlocal 用于什么？

参考答案：在嵌套函数中绑定外层（非全局）变量。

周末交付物

提交 1 页错题复盘：错误现象、最小反例、根因、正确规则、重新写出的代码或口述答案。每周至少完成 3 道代码题并写出时间/空间复杂度。

第 4 周：面向对象与准确表达

掌握类与实例的边界，能用 `self/cls` 解释三类方法，并写出继承和重写。

本周知识点

- 类、实例、实例属性与类属性
- `self`、`cls` 的含义与绑定
- 实例方法、`@classmethod`、`@staticmethod`
- `@property` 的受控访问
- 继承、方法重写、`super()` 与多态

每日执行任务

日期	主题	完成动作
Day 1	类与实例	定义 <code>Student</code> ，创建两个对象；比较实例属性与类属性的共享方式。
Day 2	三种方法	为 <code>Temperature</code> 分别写实例转换、类构造、静态校验方法。
Day 3	<code>property</code>	用私有 <code>_score</code> 和 <code>@property</code> 校验分数 0 到 100。
Day 4	继承	创建 <code>Animal</code> 和 <code>Dog</code> ；重写 <code>speak</code> ，并调用 <code>super().__init__</code> 。
Day 5	多态	让不同子类在同一循环中执行相同接口。
Day 6	口述训练	用 60 秒完整回答实例/类/静态方法的区别，录音后自查。
Day 7	阶段测验 2	完成 OOP 小题与代码题；根据评分表复盘。

代码练习

- 1 实现 `Person.from_string(cls, text)`，作为替代构造方法。
- 2 实现 `BankAccount` 的 `deposit/withdraw` 和 `balance` property，限制非法金额。
- 3 实现 `Shape`、`Circle`、`Rectangle`，重写 `area()` 并用同一函数求总面积。

面试问答卡

问题	最佳回答要点
实例方法、类方法、静态方法如何一句话区分？	<code>self</code> 操作具体对象； <code>cls</code> 操作类；静态方法不依赖实例或类状态，只是逻辑上归类。
为什么子类初始化常用 <code>super().__init__</code> ？	复用父类初始化，避免重复代码，并让父类需要的状态正确建立。

易错点

- 把类属性误当成每个对象独有，或通过实例修改后产生遮蔽。
- 在 `@property` setter 中给 `self.score` 赋值造成递归。
- 重写 `__init__` 后忘记调用父类初始化。

本周自测与参考答案

1. 题目：`classmethod` 第一个参数通常是什么？
参考答案：`cls`，表示当前类；子类调用时会自动绑定子类。
2. 题目：静态方法能访问 `self` 吗？

参考答案：不能自动获得 `self`；需要显式传入对象才可以使用。

3. 题目：property 的价值是什么？

参考答案：外部仍可用属性语法，同时在内部加入校验、计算或兼容逻辑。

周末交付物

提交 1 页错题复盘：错误现象、最小反例、根因、正确规则、重新写出的代码或口述答案。每周至少完成 3 道代码题并写出时间/空间复杂度。

第 5 周：复杂度、哈希与基础结构

写代码时能说明复杂度，并根据约束选择字典、双指针、滑动窗口、栈和队列。

本周知识点

- 时间/空间复杂度的基本估算
- 列表、字典、集合的常见操作成本
- 哈希表计数与查找
- 双指针与滑动窗口
- 栈 LIFO、队列 FIFO 与 deque

每日执行任务

日期	主题	完成动作
Day 1	复杂度	给 10 段代码标注 $O(1)$ 、 $O(n)$ 、 $O(n \log n)$ 、 $O(n^2)$ 。
Day 2	字典哈希	实现出现次数最多元素；比较排序法与一次遍历法。
Day 3	双指针	完成有序数组两数之和、原地移除元素。
Day 4	滑动窗口	完成无重复字符最长子串的基础版本。
Day 5	栈	验证括号匹配；解释后进先出。
Day 6	队列	用 <code>collections.deque</code> 写 BFS 的入队/出队骨架。
Day 7	周复盘	完成自测；给第 5 题的两种方案写复杂度说明。

代码练习

- 1 实现 `most_frequent(nums)`：次数相同返回首次出现者，空列表返回 `None`，整体 $O(n)$ 。
- 2 实现 `two_sum_sorted(nums, target)`，返回一对下标或 `None`。
- 3 实现 `is_valid_brackets(text)`，只处理 `()[]{}` 并用栈匹配。

面试问答卡

问题	最佳回答要点
字典查找为什么通常是 $O(1)$ ？	哈希表根据哈希值定位桶，平均不需线性扫描；极端冲突时会变差。
什么时候滑动窗口适合？	连续子数组/子串，且窗口扩张和收缩能维护条件时。

易错点

- 把 `sorted()` 误认为 $O(n)$ 。
- 循环内用 `list.count` 导致隐藏 $O(n^2)$ 。
- BFS 用 `list.pop(0)` 造成 $O(n^2)$ 出队。

本周自测与参考答案

1. 题目：遍历 n 个元素再排序的复杂度？
参考答案： $O(n \log n)$ ，排序主导。
2. 题目：栈与队列的访问端分别是什么？
参考答案：栈同端进出，LIFO；队列一端入另一端出，FIFO。

3. 题目：最常见的哈希计数空间复杂度？

参考答案： $O(k)$ ， k 为不同元素个数，最坏 $O(n)$ 。

周末交付物

提交 1 页错题复盘：错误现象、最小反例、根因、正确规则、重新写出的代码或口述答案。每周至少完成 3 道代码题并写出时间/空间复杂度。

第 6 周：常见算法与面试整合

建立可迁移的算法框架，完成复盘、阶段测验 3 与最终模拟面试。

本周知识点

- 二分查找的区间与循环不变式
- 递归的终止条件与递归关系
- DFS/BFS 的访问标记
- 排序、贪心与动态规划入门
- 面试表达：澄清、方案、复杂度、测试、改进

每日执行任务

日期	主题	完成动作
Day 1	二分查找	写闭区间二分，明确 left/right 更新；测试空、单元素、未命中。
Day 2	递归	写阶乘和二叉树深度伪代码；每题先写 base case。
Day 3	DFS/BFS	在网格中统计连通块；比较递归 DFS 与队列 BFS。
Day 4	排序/贪心	解释排序复杂度；完成区间选择或找零的贪心反例分析。
Day 5	DP 入门	完成爬楼梯，写状态定义、转移方程、初始值。
Day 6	全真模拟	按最终清单完成 5 题，限时 45 分钟并口述解题过程。
Day 7	复盘与下一轮	完成阶段测验 3，整理高频错误并制定下 4 周巩固题单。

代码练习

- 1 实现 `binary_search(nums, target)`，返回下标或 `-1`，并说明循环不变式。
- 2 实现 `count_islands(grid)` 的 DFS/BFS 版本，避免重复访问。
- 3 实现 `climb_stairs(n)`，先写递归关系，再优化为 $O(1)$ 额外空间。

面试问答卡

问题	最佳回答要点
二分为何要求有序或单调？	只有单调性才能根据一次比较排除半边搜索空间。
DP 与递归的关系？	DP 将有重叠子问题的递归结果保存，避免重复计算；可自顶向下记忆化或自底向上。

易错点

- 二分 `left/right` 不更新导致死循环或漏掉边界。
- 递归没有 `base case`。
- BFS 未在入队时标记，导致重复入队。

本周自测与参考答案

1. 题目：二分搜索 `[1, 3, 5]` 中 `2`，最终为什么返回 `-1`？
参考答案：循环结束说明候选区间为空，未找到目标。
2. 题目：DFS 与 BFS 的核心数据结构？
参考答案：DFS 常用递归调用栈或显式栈；BFS 用队列。

3. 题目：爬楼梯的状态转移？

参考答案： $dp[i]=dp[i-1]+dp[i-2]$ ，最后一步来自 $i-1$ 或 $i-2$ 。

周末交付物

提交 1 页错题复盘：错误现象、最小反例、根因、正确规则、重新写出的代码或口述答案。每周至少完成 3 道代码题并写出时间/空间复杂度。

关键代码模板

模板不能替代理解。使用时请先说出输入、边界、循环不变式和复杂度。

1. 质数：正确处理边界与循环返回

```
def is_prime(number):
    if number < 2:
        return False
    for i in range(2, int(number ** 0.5) + 1):
        if number % i == 0:
            return False
    return True
```

解析：True 必须在循环后返回；只检查到平方根即可。时间 $O(\sqrt{n})$ ，空间 $O(1)$ 。

2. 可变默认参数：None 哨兵

```
def add_item(item, container=None):
    if container is None:
        container = []
    container.append(item)
    return container
```

解析：None 不会与合法的空列表混淆。不要写 `if not container`，因为调用者传 `[]` 时也会被误判。

3. 并列取首次出现的众数： $O(n)$

```
def most_frequent(nums):
    if not nums:
        return None
    counts = {}
    first = {}
    for index, value in enumerate(nums):
        counts[value] = counts.get(value, 0) + 1
        if value not in first:
            first[value] = index

    best = None
    for value in counts:
        if (best is None or counts[value] > counts[best] or
            (counts[value] == counts[best] and first[value] < first[best])):
            best = value
    return best
```

解析：两次均为 $O(n)$ ，字典空间最坏 $O(n)$ 。面试时务必先确认并列规则。

4. for...else 搜索

```
def find_first(nums, target):
    for index, value in enumerate(nums):
        if value == target:
            return index
    else:
        return -1
```

解析：没有 break 自然结束才执行 else。此处也可以把 `return -1` 放在循环外，语义更常见。

题库 A：20 道选择题

使用方法：先遮住答案，限时 20 分钟完成。每题错了都要写出“错误选项为何诱人”。

1. [易] 执行 `a=[1]`；`b=a`；`b.append(2)` 后 `a` 的值是？

- A. [1] B. [1, 2] C. [2] D. 报错

答案：B。解析：a 与 b 引用同一可变列表，append 原地修改对象。

2. [易] 下面函数连续调用 `f(1)`、`f(2)` 的默认输出特征是？`def f(x, a=[]): a.append(x); return a`

- A. 独立列表 B. 共享同一默认列表 C. 报 `SyntaxError` D. 返回元组

答案：B。解析：默认列表在函数定义时创建一次，未传 a 的调用会共享它。

3. [易] 下列哪个是不可变类型？

- A. list B. dict C. set D. tuple

答案：D。解析：tuple 本身不可变；若包含可变对象，内部对象仍可变。

4. [易] 循环中的 `break` 会？

- A. 结束整个函数 B. 跳过本次迭代 C. 结束当前循环 D. 返回 None

答案：C。解析：break 仅跳出最近的一层循环；return 才结束函数。

5. [易] Python 方法中 `self` 指向？

- A. 类 B. 当前实例 C. 父类 D. 模块

答案：B。解析：实例调用实例方法时，实例会自动绑定给 self。

6. [易] 字典按键查找平均时间复杂度通常是？

- A. $O(1)$ B. $O(\log n)$ C. $O(n)$ D. $O(n \log n)$

答案：A。解析：哈希表平均可直接定位桶；不承诺所有极端情况。

7. [易] `for...else` 的 `else` 通常在何时执行？

- A. 首次循环 B. 条件为假 C. 循环未被 break 打断并自然结束 D. 一定执行

答案：C。解析：break 会跳过循环 else。

8. [中] `copy.copy` 对嵌套列表的行为是？

- A. 递归复制全部层 B. 只复制外层容器 C. 不创建新对象 D. 将列表转元组

答案：B。解析：浅拷贝创建新外层，内层对象引用仍共享。

9. [中] 使用 `None` 作为列表默认参数哨兵的主要原因是？

- A. `None` 更快 B. `None` 会自动变列表 C. 区分未传参与合法空列表 D. 防止类型检查

答案：C。解析：if container is None 只处理未传入的情形，保留调用者的 []。

10. [中] `@classmethod` 的第一个参数通常是？

- A. self B. cls C. class D. obj

答案：B。解析：cls 是当前类，适合替代构造或操作类级状态。

11. [中] 下列哪一项最适合 `staticmethod`？

- A. 读取实例余额 B. 修改类计数器 C. 校验身份证格式的工具逻辑 D. 创建实例并读取 self

答案：C。解析：静态方法不需要自动的 self/cls，但逻辑上与类相关。

12. [中] 二分查找适用的前提是？

- A. 数据无序 B. 存在有序或单调性 C. 必须是字符串 D. 长度为偶数

答案：B。解析：比较结果能够排除一半空间依赖序列有序或判定单调。

13. [中] 两层各遍历 `n` 次的嵌套循环通常复杂度是？

- A. $O(1)$ B. $O(\log n)$ C. $O(n)$ D. $O(n^2)$

答案：D。解析：总执行次数约为 n^2 。

14. [中] 滑动窗口最典型的对象是？

- A. 任意散点集合 B. 连续子数组或子串 C. 树的层序遍历 D. 类继承

答案：B。解析：窗口维护的是连续区间，并按条件扩张或收缩。

15. [中] LEGB 中 E 表示？

- A. Error B. External C. Enclosing D. Expression

答案：C。解析：名称查找顺序为 Local、Enclosing、Global、Builtins。

16. [难] 以下二分循环若使用闭区间 $[left, right]$, $target > nums[mid]$ 时正确更新是？

A. $left=mid$ B. $left=mid+1$ C. $right=mid$ D. $right=mid-1$

答案：B。解析： mid 已检查且更小，必须排除，左边界移动到 $mid+1$ 。

17. [难] 递归函数最重要的安全条件是？

A. 使用 `global` B. 必须返回列表 C. 有可达到的终止条件 D. 参数必须为整数

答案：C。解析：没有 `base case` 或无法到达会无限递归并触发 `RecursionError`。

18. [难] BFS 访问图节点时，通常何时标记 `visited` 更稳妥？

A. 出队后 B. 入队时 C. 全部结束后 D. 从不标记

答案：B。解析：入队即标记可避免同一节点被多个邻居重复加入队列。

19. [难] 爬楼梯 $dp[i]=dp[i-1]+dp[i-2]$ 的含义是？

A. 当前位置只能走一步 B. 最后一步来自两种前一状态 C. 必须排序 D. 贪心总是最优

答案：B。解析：到第 i 阶的路径按最后走 1 阶或 2 阶划分。

20. [难] 若一个算法每个元素入栈、出栈各至多一次，其时间复杂度通常为？

A. $O(1)$ B. $O(\log n)$ C. $O(n)$ D. $O(n^2)$

答案：C。解析：摊还来看每个元素只被常数处理。

题库 B：15 道简答题

建议每题用 60 到 90 秒作答。结构：结论 -> 机制 -> 最小示例或场景。

1. [易] 解释变量、对象和引用的关系。

最佳答案：变量名绑定对象；多个变量可引用同一对象。赋值通常复制引用，不自动复制对象。

2. [易] return、break、continue 的区别是什么？

最佳答案：return 结束函数并返回值；break 结束当前循环；continue 跳过本轮循环余下语句，进入下一轮。

3. [易] 为什么 0 和 1 不是质数？

最佳答案：质数定义为大于 1 且只有 1 和自身两个正因子的整数；0 和 1 不满足。

4. [易] 为什么要测试空列表？

最佳答案：空输入常会暴露索引越界、除零、未初始化最优值等问题；必须明确返回或异常约定。

5. [易] 列出三种常见边界测试。

最佳答案：空输入、单元素、重复元素；还可测试最小/最大值、负数、未命中和非法类型。

6. [中] 浅拷贝和深拷贝的区别及使用场景。

最佳答案：浅拷贝只复制外层，嵌套对象共享，适合元素不可变或无需改嵌套项；深拷贝递归复制，适合需要完全独立的嵌套可变数据。

7. [中] 为什么可变默认参数危险？如何修复？

最佳答案：默认对象在定义时创建一次，调用之间会共享状态。默认 None，在函数内 if 参数 is None 时新建对象。

8. [中] 实例属性和类属性如何区分？

最佳答案：实例属性通常在 self 上，属于单个对象；类属性定义在类体，默认被实例共享。通过实例赋值同名属性会遮蔽类属性。

9. [中] 说明实例方法、类方法、静态方法及各一个用途。

最佳答案：实例方法接收 self、操作实例状态，如存款；类方法接收 cls、替代构造或操作类状态；静态方法不依赖二者，如格式校验。

10. [中] 什么是 LEGB？

最佳答案：名称查找从局部 Local 到闭包 Enclosing、全局 Global、内建 Builtins；找到即停止。

11. [中] 为什么字典统计频次常优于 list.count 循环？

最佳答案：字典单次遍历累计平均 $O(n)$ ；每个元素调用 count 会反复扫描列表，最坏 $O(n^2)$ 。

12. [难] 如何向面试官说明二分查找正确性？

最佳答案：说明有序前提和循环不变式：若 target 存在，始终在当前候选区间。比较 mid 后排除不可能的一半，区间为空即未找到。

13. [难] DFS 与 BFS 的选择依据是什么？

最佳答案：DFS 适合深度探索、回溯、连通性；BFS 按层扩展，适合无权图最短步数。两者都要处理 visited。

14. [难] 贪心何时可信？

最佳答案：需要证明局部最优选择可扩展为全局最优，通常通过交换论证或贪心选择性质；不能仅凭直觉。

15. [难] 动态规划解题的四步是什么？

最佳答案：定义状态含义；写状态转移；设定初始值；确定遍历顺序和答案位置，并分析复杂度。

题库 C: 15 道编程题

每题先口述：输入输出、边界、方案、复杂度；再写代码；最后至少写 3 个测试。

1. [易 | E1] 写 `is_prime(number)`, `number < 2` 返回 `False`; 优化到平方根。

最佳思路与解析：见关键代码模板。注意 `True` 要在循环结束后返回；时间 $O(\sqrt{n})$ 。

2. [易 | E2] 写 `find_first(nums, target)`, 返回首次下标, 未找到 `-1`。

最佳思路与解析：遍历 `enumerate`, 命中立即 `return index`; 循环后 `return -1`。时间 $O(n)$, 空间 $O(1)$ 。

3. [易 | E3] 写 `add_item(item, container=None)`, 避免共享默认列表。

最佳思路与解析：参数 `None` 时新建 `[]`, 再 `append`。不能把 `[]` 作为默认值。

4. [易 | E4] 写 `remove_duplicates_keep_order(nums)`, 保留首次出现顺序。

最佳思路与解析：维护 `seen` 集合与 `result` 列表；首次未见时追加。时间 $O(n)$, 空间 $O(n)$ 。

5. [易 | E5] 写 `sum_until_negative(nums)`, 遇负数停止。

最佳思路与解析：`total=0`; 遍历时若 `x < 0`: `break`, 否则累计；返回 `total`。

6. [中 | M1] 写 `most_frequent(nums)`, 并列时返回最先出现的元素, 空列表返回 `None`。

最佳思路与解析：统计频次与首次下标, 遍历候选选择 `count` 更大或同频首位更小者。 $O(n)$ 时间、 $O(k)$ 空间。

7. [中 | M2] 有序数组写 `two_sum_sorted(nums, target)`, 返回一对下标或 `None`。

最佳思路与解析：`left=0, right=len-1`; 和小则 `left+=1`, 大则 `right-=1`, 命中返回。 $O(n)$ 。

8. [中 | M3] 写 `is_valid_brackets(text)`, 检查 `()[]{}` 配对。

最佳思路与解析：遇左括号入栈；遇右括号检查栈顶是否匹配；末尾栈空才 `True`。 $O(n)$ 时间和空间。

9. [中 | M4] 写 `binary_search(nums, target)`, 返回下标或 `-1`。

最佳思路与解析：闭区间时 `while left <= right`; 比较 `mid` 后用 `mid+1/mid-1`; 循环结束未命中 `-1`。

10. [中 | M5] 写 `longest_unique_substring_len(text)`。

最佳思路与解析：字典记录字符最后位置, `left` 只右移；遍历 `right` 更新最大窗口。 $O(n)$ 。

11. [难 | H1] 写 `Person.from_string(cls, text)`, 输入 '姓名, 年龄', 返回实例并校验年龄。

最佳思路与解析：使用 `@classmethod`; `split` 后转换年龄；非法格式抛 `ValueError`。体现替代构造。

12. [难 | H2] 写 `BankAccount`, 余额只读 `property`, `deposit/withdraw` 校验金额与余额。

最佳思路与解析：内部 `_balance`; `property` 只读；方法中金额必须 > 0 , 取款不得超余额。

13. [难 | H3] 网格 1 表示陆地, 写 `count_islands(grid)`。

最佳思路与解析：遍历网格, 遇未访问陆地计数并 DFS/BFS 标记相邻陆地； $O(mn)$ 。

14. [难 | H4] 写 `climb_stairs(n)`, 每次可 1 或 2 阶。

最佳思路与解析：定义 `dp[i]` 到第 i 阶的方法数；`dp[i]=dp[i-1]+dp[i-2]`；可滚动变量 $O(1)$ 空间。

15. [难 | H5] 给非空区间列表, 按结束时间选择最多不重叠区间。

最佳思路与解析：按结束时间排序, 依次选 `start >= last_end` 的区间。需说明这是可交换证明成立的经典贪心。 $O(n \log n)$ 。

阶段测验、评分标准与学习工具

每次测验均记录用时。允许查资料的练习不计入测验成绩；测验后必须复盘。

阶段测验 1：第 2 周末（30 分，35 分钟）

- 1 解释 `b=a` 与 `b=a.copy()` 的区别，并用嵌套列表举例。（8 分）
- 2 修复一个可变默认参数函数，并解释为何 `if x is None`。（8 分）
- 3 编写函数判断二维列表是否存在共享行，并给出正确创建方式。（8 分）
- 4 写出 4 个边界测试及预期。（6 分）

参考要点

必须出现：赋值共享对象；浅拷贝外层独立/内层共享；None 区分未传与空列表；`[[0]*m for _ in range(n)]`；边界包括空、单、嵌套修改、非法输入。

阶段测验 2：第 4 周末（40 分，45 分钟）

- 1 用表格比较实例方法、类方法、静态方法，并各给一个场景。（10 分）
- 2 实现带 `score` property 的 `Student`，限定 0 到 100。（12 分）
- 3 实现父类 `Animal` 与子类 `Dog`，重写 `speak`，正确使用 `super`。（10 分）
- 4 解释实例属性遮蔽类属性的现象。（8 分）

参考要点

方法题须说明 `self/cls`/无自动参数；property setter 使用 `_score` 避免递归；子类初始化调用 `super().__init__`；实例同名赋值不会修改类属性。

阶段测验 3：第 6 周末（50 分，60 分钟）

- 1 写二分查找并说明循环不变式与三个边界测试。（12 分）
- 2 写最多频率元素并列取首出现者，给 $O(n)$ 解法。（12 分）
- 3 写括号匹配或岛屿数量二选一，并说明数据结构。（14 分）
- 4 为一段含排序的代码写时间和空间复杂度。（6 分）
- 5 用 90 秒讲解一次你修复提前 `return` 的过程。（6 分）

参考要点

二分要排除 `mid`；频次题要处理空输入和并列；图题要避免重复访问；复杂度表述包含 n/k 的含义；讲解要给最小反例，如 `number=9`。

统一评分标准

维度	评分要点	占比
正确性	正常案例与边界案例均正确	40%
思路与表达	先澄清约定，能说明为何这样做	25%
代码质量	命名、缩进、函数边界与异常约定清晰	15%

维度	评分要点	占比
复杂度	能给出时间/空间复杂度，并识别可优化点	10%
测试意识	主动给出空、单元素、重复、临界等测试	10%

成绩解释

90-100：能稳定面试表达并处理边界；75-89：基础扎实，继续提升算法熟练度；60-74：能写部分代码，但需强化执行推演与概念表达；60 以下：回到错题重做与口述训练。

错题记录、每日打卡与最终模拟面试

把“错一次”变成“以后不会再错”的核心，是记录具体反例，而不是只抄正确答案。

错题记录模板（每题一页或一条）

字段	填写内容
题目与来源	题号、日期、题目链接或截图位置
我的错误答案	保留原始思路，不要改写
最小反例	能立刻暴露错误的最小输入，例如 <code>is_prime(9)</code>
错误根因	缩进、循环提前返回、默认参数、复杂度误判、概念表达等
正确规则	用一句自己的话写出可复用规则
重做日期	24 小时后、7 天后、21 天后各复做一次

每日打卡表（可打印）

日期	学习 30 分	代码 30 分	口述 10 分	错题复盘	完成度/备注
第 __ 周 Day 1	☐	☐	☐	☐	_____
第 __ 周 Day 2	☐	☐	☐	☐	_____
第 __ 周 Day 3	☐	☐	☐	☐	_____
第 __ 周 Day 4	☐	☐	☐	☐	_____
第 __ 周 Day 5	☐	☐	☐	☐	_____
第 __ 周 Day 6	☐	☐	☐	☐	_____
第 __ 周 Day 7	☐	☐	☐	☐	_____

最终模拟面试清单（45 分钟）

环节	检查项	建议用时
开始	复述题意；确认输入、输出、并列/异常规则	每题 1 分钟
分析	先给朴素解，再给优化解；说明取舍	每题 2-3 分钟
编码	函数命名清晰；关键边界提前处理；避免循环内错误 <code>return</code>	每题 4-6 分钟
测试	至少说出空、单元素、重复、临界和一个反例	每题 1 分钟
收尾	报告时间/空间复杂度及可能改进	每题 30 秒

建议模拟题顺序

- 代码输出：引用/浅拷贝 + 可变默认参数。
- 概念表达：实例方法、类方法、静态方法；再追问 `property` 与 `super`。
- 代码修复：循环中提前 `return` 的质数函数。
- 编程：最多频率元素，并列取最先出现；追问 $O(n)$ 优化。
- 算法：二分或括号匹配；追问边界与复杂度。

6 周结束后：4 周巩固循环

完成计划后，不要立刻跳到大量难题。以“间隔复习 + 限时表达”把知识变成稳定能力。

巩固周	训练重点	每周成果
第 7 周	重做全部错题；不看答案口述 10 个核心概念	错题正确率达到 85% 以上
第 8 周	每天 1 道哈希/双指针/栈队列题，限时 25 分钟	每题有边界与复杂度说明
第 9 周	每天 1 道二分/递归/DFS-BFS/DP 入门题	至少完成 4 次白板式讲解
第 10 周	两次完整模拟面试，回看录音并修订表达	形成自己的高频问答卡

每天 15 分钟保温法

- 3 分钟：随机抽一张概念卡，按“结论 -> 机制 -> 示例”说出来。
- 7 分钟：读一段短代码，写出变量变化、返回点和一个边界输入。
- 5 分钟：回顾当天的一个错题，遮住答案重新写或重新讲。

你的面试回答模板

- 我先确认一下：空输入、重复元素和并列情况怎么处理？
 - 朴素方案是……，时间复杂度为……。
 - 可以用……优化到……，核心原因是……。
 - 我会先处理边界：……；主循环保持……这个不变式。
 - 我会用……、……、……三个案例测试。

最后提醒

真实面试不需要一开始就写最优代码。高质量表现是：先确认规则，再说方案与复杂度，边写边处理边界，最后用反例验证。你当前的优势在于能完成字典统计与排序；把解释与推演补齐，进步会很快。